The Tricks You Must Know

Erisa Terolli

Slide Credits to Shusen Wang

The Dogs vs. Cats Dataset

The Dogs vs. Cats Dataset



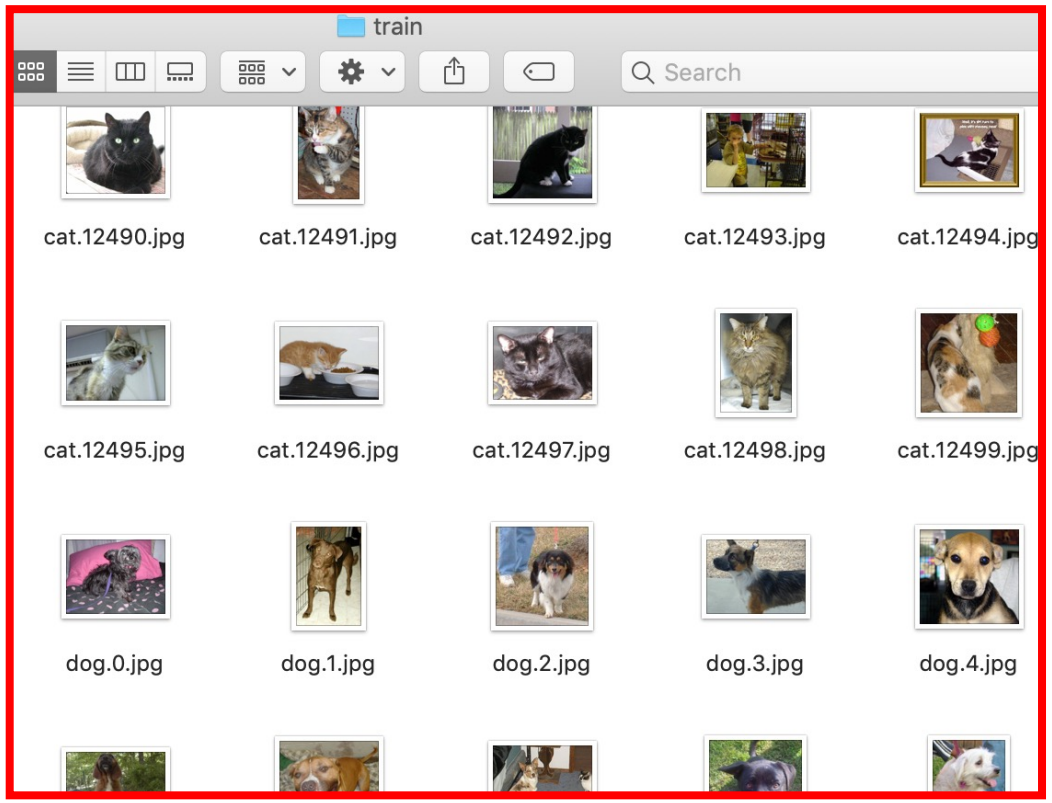


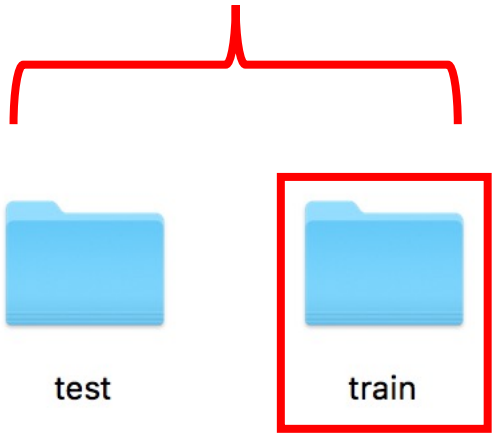




The Dogs vs. Cats Dataset

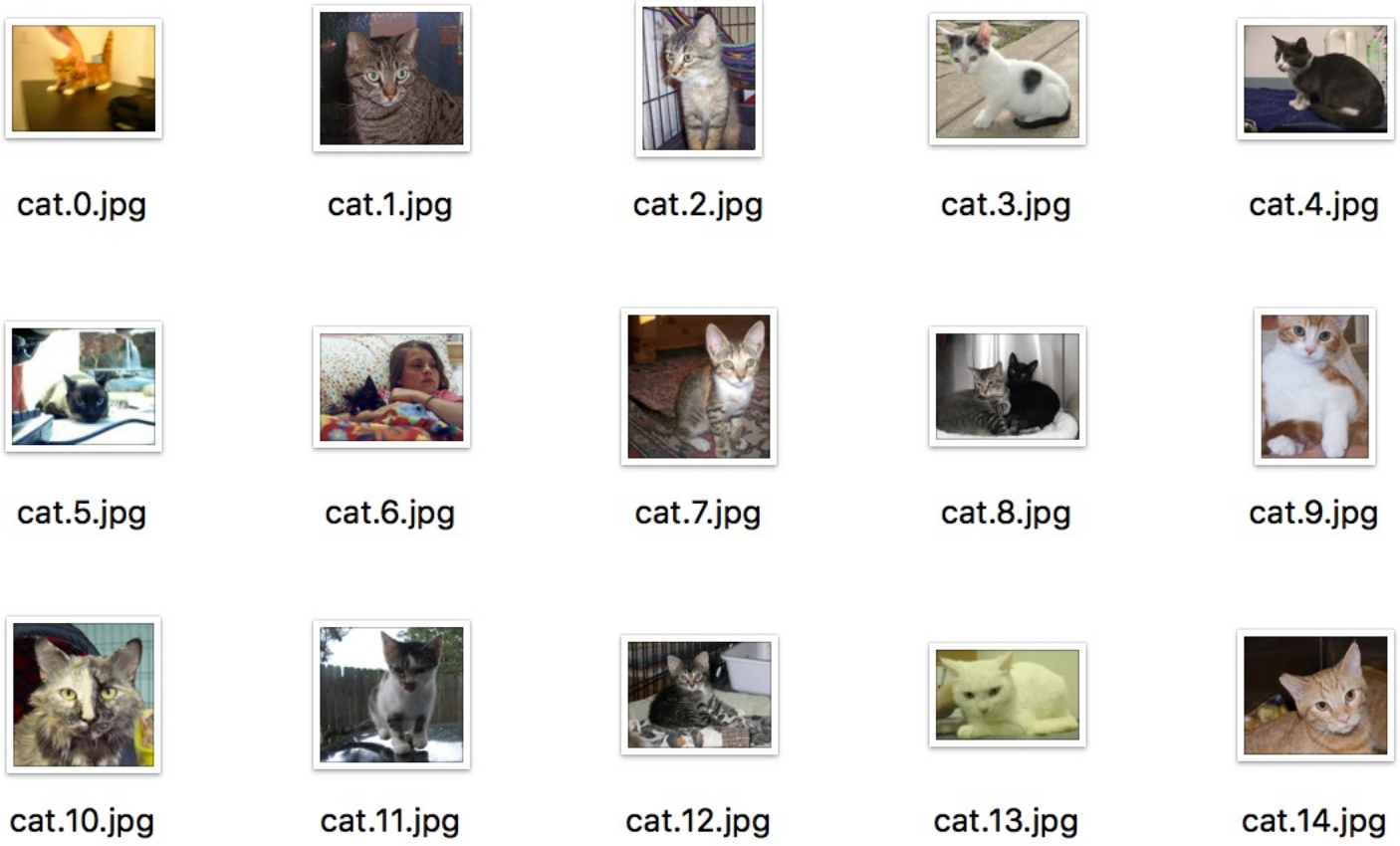
Download: https://www.kaggle.com/c/dogs-vs-cats/data

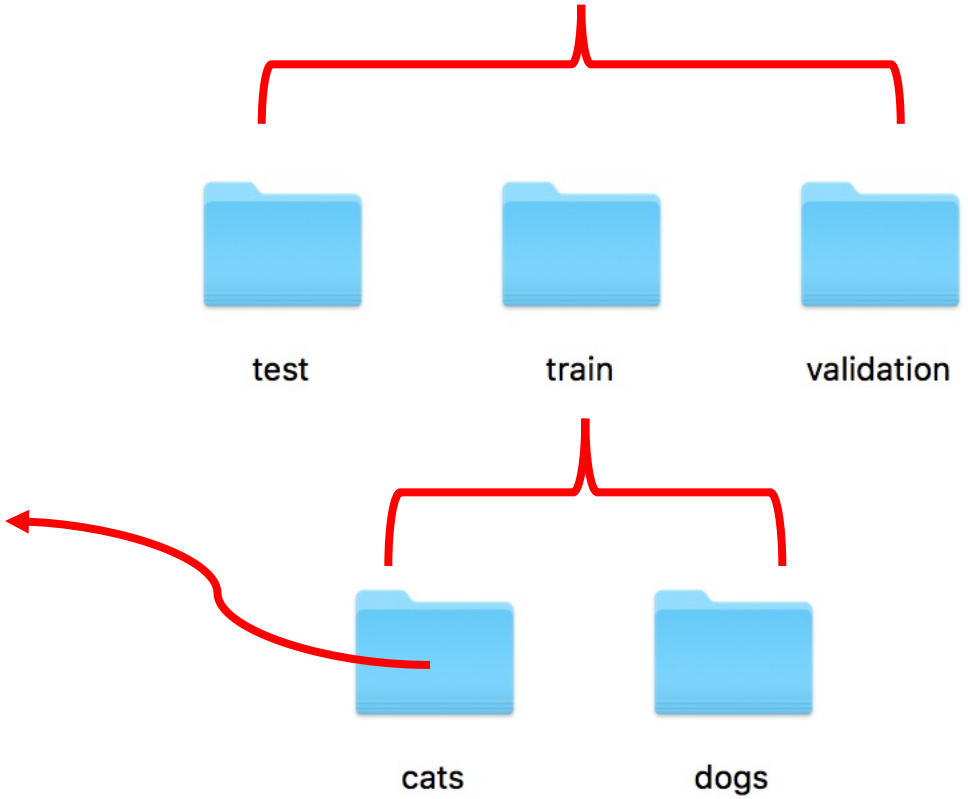




The Dogs vs. Cats Dataset

Download: https://www.kaggle.com/c/dogs-vs-cats/data





The Dogs vs. Cats Dataset

- The dataset has 25,000 training samples.

- I use a subset:
  - 2000 images for training,
  - 1000 images for validation,
  - 1000 images for test.

Implement a CNN Using Keras

1. Load and Process the Dataset

- Currently, the data sit on a drive as JPEG files.
- Data processing:
  1. Read the picture files.
  2. Decode the JPEG content to order-3 tensors.
  3. Resize the images to the the same shape, e.g., $150 \times 150 \times 3$.
  4. Rescale the pixel values (between 0 and 255) to the [0, 1] interval.

1. Load and Process the Dataset

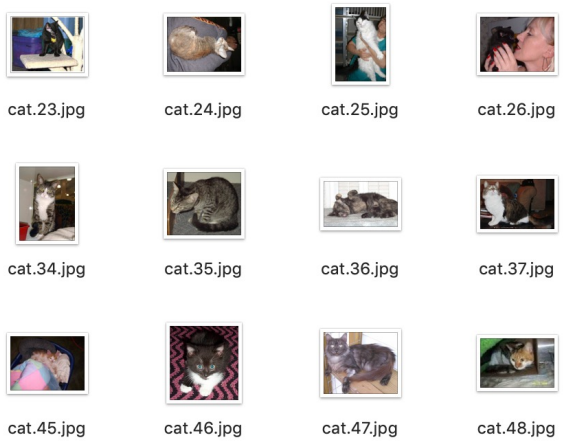
```
from  keras.preprocessing import image_dataset_from_directory
train_generator = image_dataset_from_directory(
    train_dir,
    batch_size=20,
    image_size=(150, 150),
    label_mode = "binary",
    class_names = ["cats", "dogs"]
)
validation_generator = image_dataset_from_directory(
    validation_dir,
    batch_size=20,
    image_size=(150, 150),
    label_mode = "binary",
    class_names = ["cats", "dogs"]
)
```

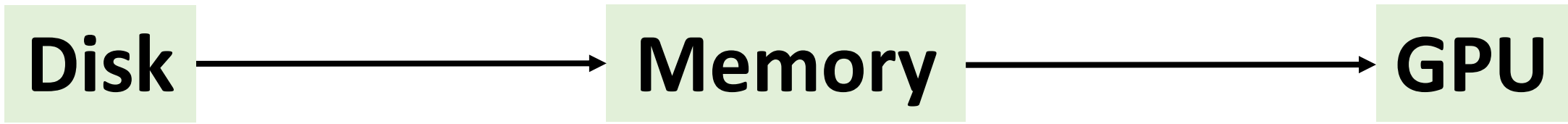
1. Load and Process the Dataset

```
for data_batch, labels_batch in train_generator:
    print('data batch shape:', data_batch.shape)
    print('labels batch shape:', labels_batch.shape)
    break
```

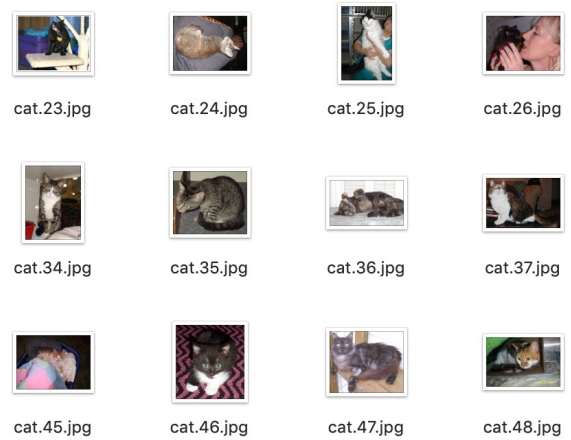
```
data batch shape: (20, 150, 150, 3)
labels batch shape: (20,)
```

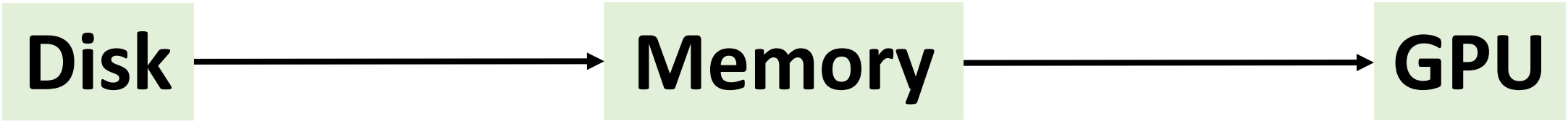
1. Load and Process the Dataset





1. Load and Process the Dataset





2. Build the CNN

```
from keras import layers
from keras import models

model = models.Sequential()
model.add(layers.Conv2D(32, (3, 3), activation='relu',
                        input_shape=(150, 150, 3)))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(128, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(128, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Flatten())
model.add(layers.Dense(512, activation='relu'))
model.add(layers.Dense(1, activation='sigmoid'))
```

2. Build the CNN

```
model.summary()
```

```
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
conv2d_1 (Conv2D)            (None, 148, 148, 32)      896
_________________________________________________________________
max_pooling2d_1 (MaxPooling2 (None, 74, 74, 32)        0
_________________________________________________________________
conv2d_2 (Conv2D)            (None, 72, 72, 64)        18496
_________________________________________________________________
max_pooling2d_2 (MaxPooling2 (None, 36, 36, 64)        0
_________________________________________________________________
conv2d_3 (Conv2D)            (None, 34, 34, 128)       73856
_________________________________________________________________
max_pooling2d_3 (MaxPooling2 (None, 17, 17, 128)       0
_________________________________________________________________
conv2d_4 (Conv2D)            (None, 15, 15, 128)       147584
_________________________________________________________________
max_pooling2d_4 (MaxPooling2 (None, 7, 7, 128)         0
_________________________________________________________________
flatten_1 (Flatten)          (None, 6272)              0
_________________________________________________________________
dense_1 (Dense)              (None, 512)               3211776
_________________________________________________________________
dense_2 (Dense)              (None, 1)                 513
=================================================================
Total params: 3,453,121
Trainable params: 3,453,121
Non-trainable params: 0
_________________________________________________________________
```

3. Train the CNN

Specify: optimization method, learning rate (LR), loss function, and metric.

```
from keras import optimizers

model.compile(loss='binary_crossentropy',
              optimizer=optimizers.RMSprop(lr=1e-4),
              metrics=['acc'])
```

3. Train the CNN

```
history = model.fit(
    train_generator,
    epochs = 30,
    validation_data = validation_generator,
)
```

3. Train the CNN

```
history = model.fit(
    train_generator,
    epochs = 30,
    validation_data = validation_generator,
)
```

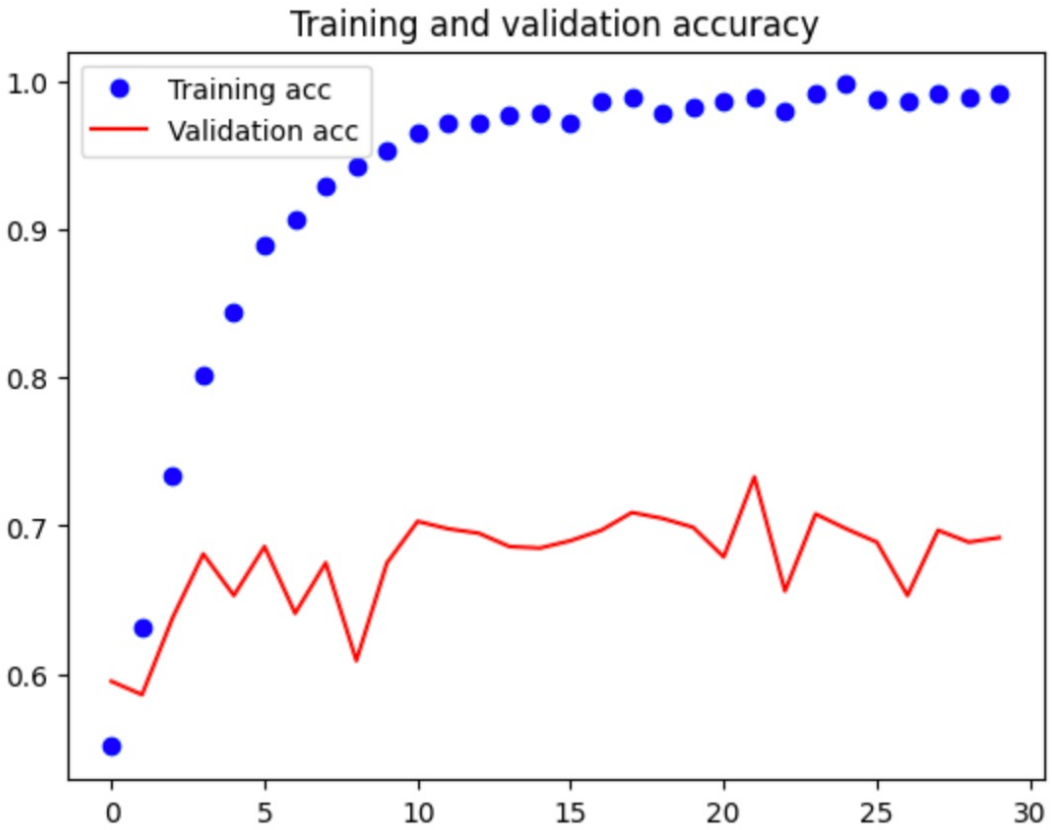
```
Epoch 1/30
100/100 ━━━━━━━━━━━━━━━━━━━━ 35s 340ms/step - acc: 0.5298 - loss: 9.4611 - val_acc: 0.5950 - val_loss: 0.6977
Epoch 2/30
100/100 ━━━━━━━━━━━━━━━━━━━━ 34s 337ms/step - acc: 0.6334 - loss: 0.6694 - val_acc: 0.5860 - val_loss: 0.6862
Epoch 3/30
100/100 ━━━━━━━━━━━━━━━━━━━━ 34s 343ms/step - acc: 0.7284 - loss: 0.5447 - val_acc: 0.6380 - val_loss: 0.7378
Epoch 4/30
100/100 ━━━━━━━━━━━━━━━━━━━━ 35s 352ms/step - acc: 0.7810 - loss: 0.4300 - val_acc: 0.6810 - val_loss: 0.7048
Epoch 5/30
100/100 ━━━━━━━━━━━━━━━━━━━━ 36s 361ms/step - acc: 0.8499 - loss: 0.3433 - val_acc: 0.6530 - val_loss: 0.8238
```

●
●
●

```
Epoch 28/30
100/100 ━━━━━━━━━━━━━━━━━━━━ 36s 356ms/step - acc: 0.9926 - loss: 0.0372 - val_acc: 0.6970 - val_loss: 1.9339
Epoch 29/30
100/100 ━━━━━━━━━━━━━━━━━━━━ 36s 355ms/step - acc: 0.9879 - loss: 0.0547 - val_acc: 0.6890 - val_loss: 1.8057
Epoch 30/30
100/100 ━━━━━━━━━━━━━━━━━━━━ 36s 356ms/step - acc: 0.9896 - loss: 0.0376 - val_acc: 0.6920 - val_loss: 1.9245
```

4. Examine the Results

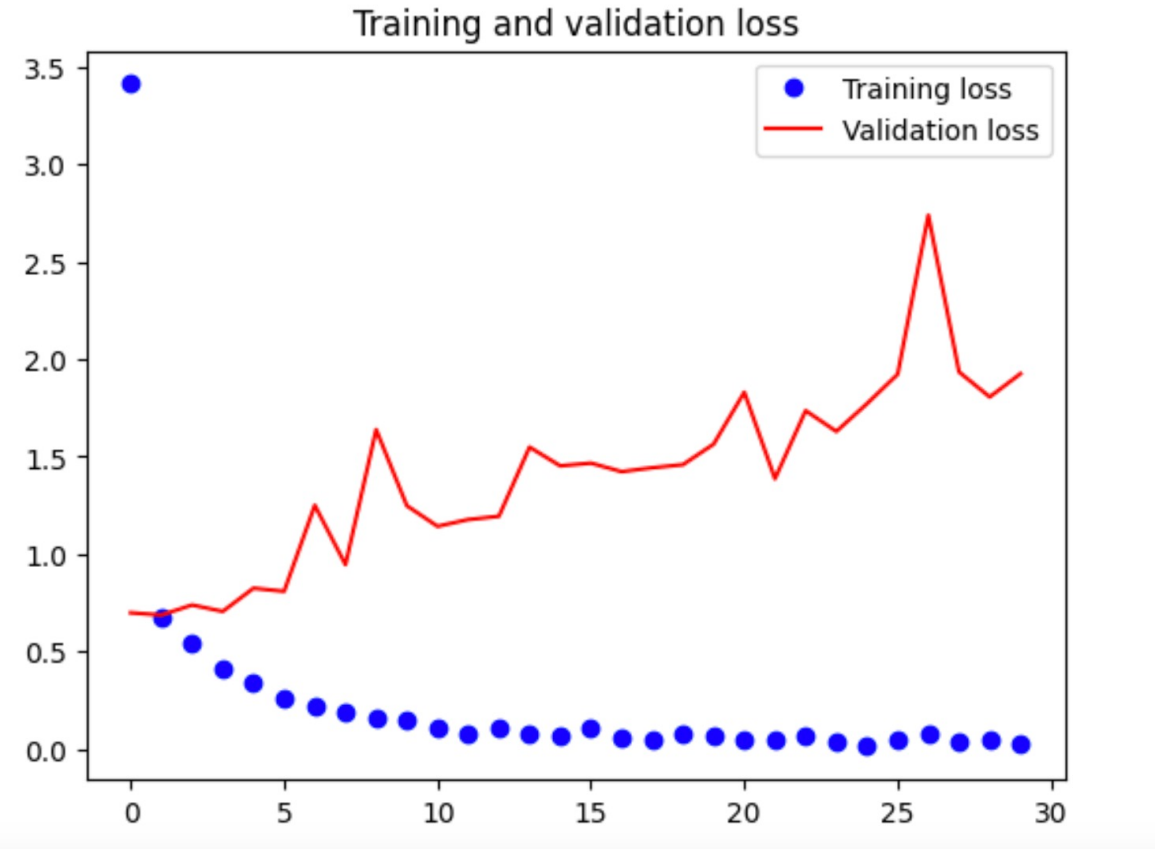
Plot the ***accuracy*** against ***epochs*** (1 epoch = 1 pass over the data).



- Training acc: 98.96%
- Validation acc: 69.2%
- Likely overfitting

4. Examine the Results

Plot the ***loss*** against ***epochs*** (1 epoch = 1 pass over the data).



- Training loss is decreasing.
- Validation loss decreases and then increase.

Why Overfitting?

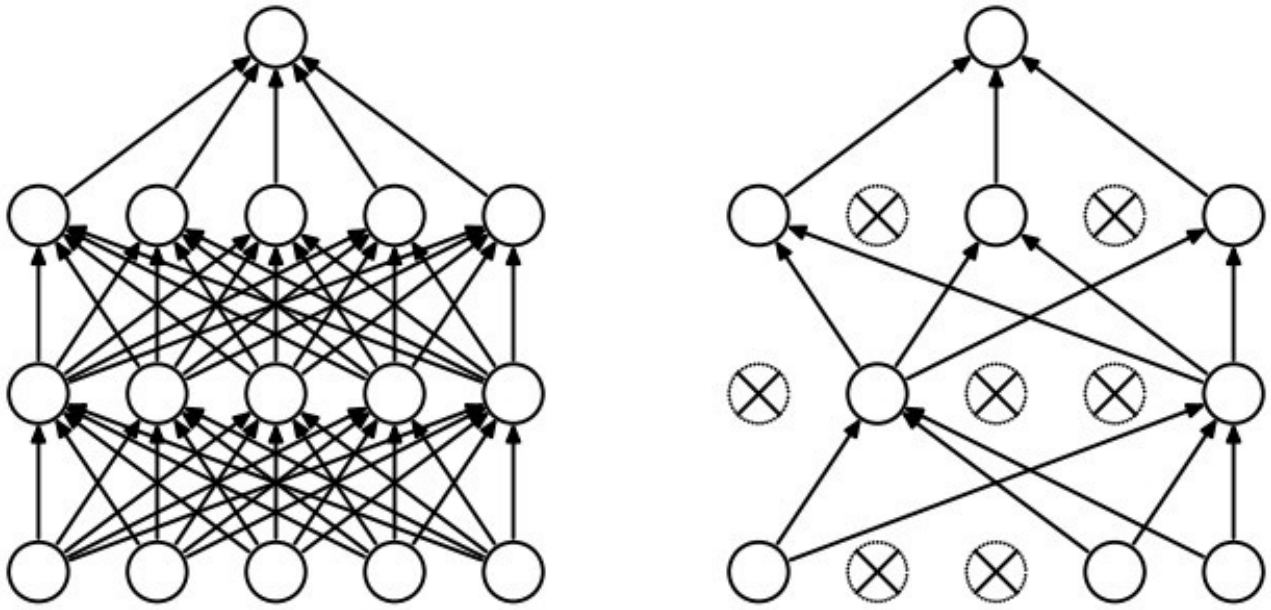
```
=================================
Total params: 3,453,121
Trainable params: 3,453,121
Non-trainable params: 0
_________________________________
```

- Over $3M$ parameters; Just $2K$ training samples.
- Overfitting is not surprising.

Trick 1: Dropout

Dropout: Basic Idea

- Train
  - In each iteration of training (1 forward + 1 backward), randomly mask 50% (or an arbitrary percentage) of the neurons.



(a) Standard Neural Net

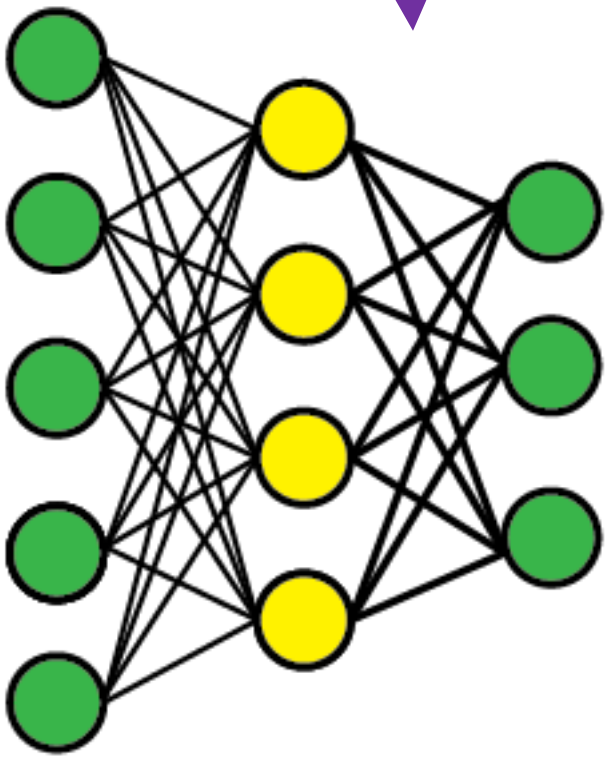
(b) After applying dropout.

Dropout: Basic Idea

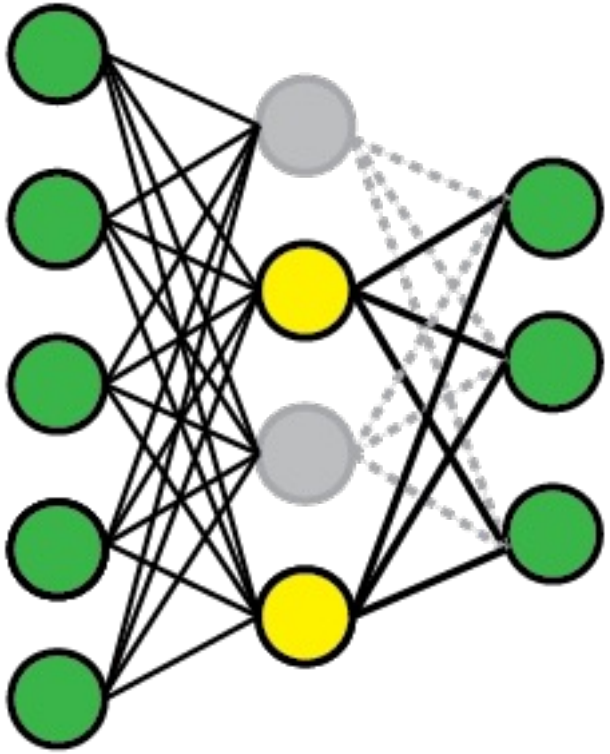
- Train
  - In each iteration of training (1 forward + 1 backward), randomly mask 50% (or an arbitrary percentage) of the neurons.
- Prediction
  - No dropout.
  - Use all the parameters.

Dropout: Implementation

Do dropout for this layer

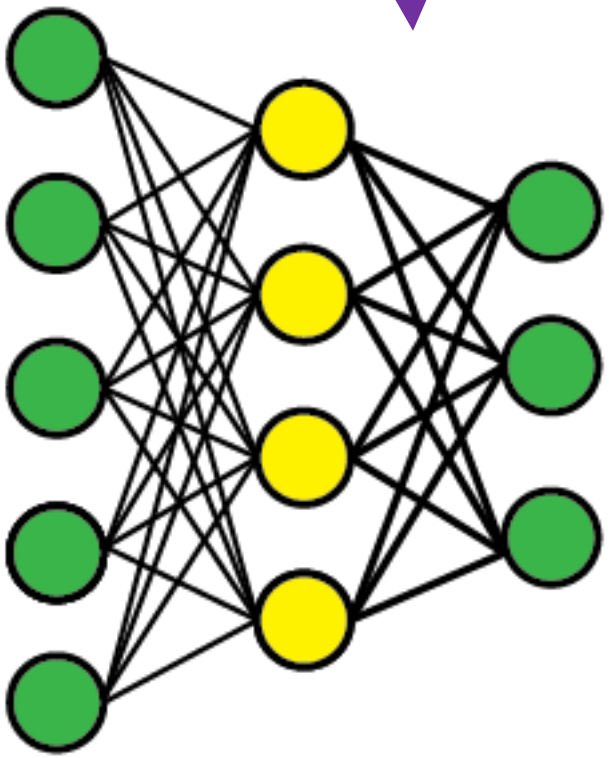


- **For a batch of training samples**
  - Randomly choose 50% of the neurons.
  - Set the selected neurons to zeros.
  - Multiply the unselected neurons by $\frac{1}{0.5} = 2$.

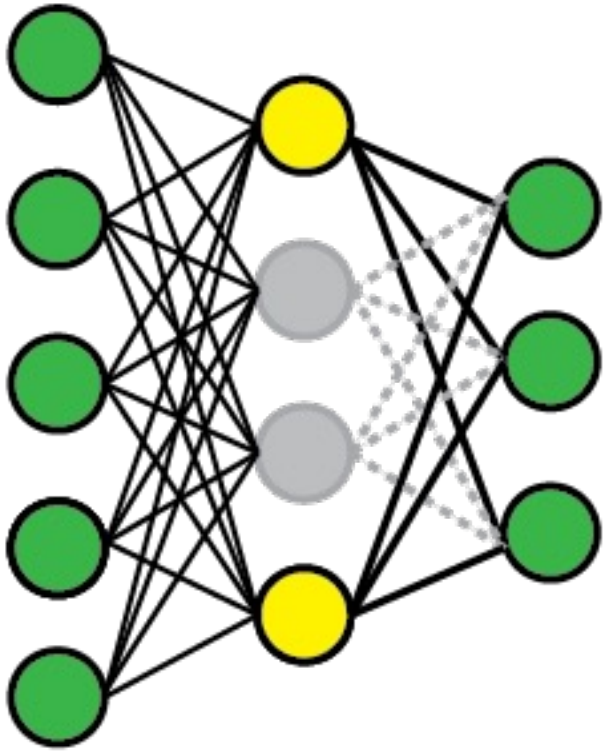


Dropout: Implementation

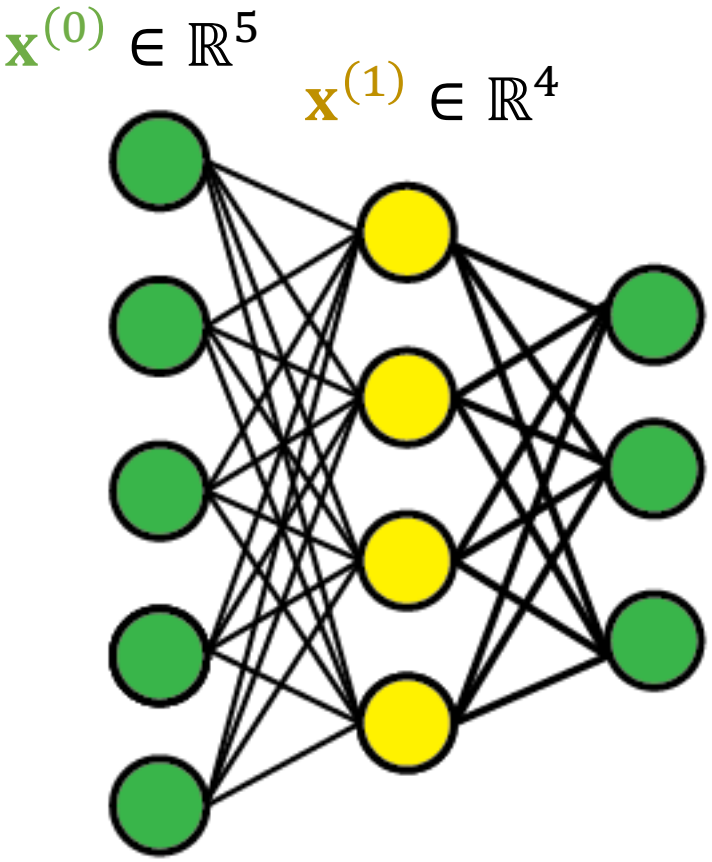
Do dropout for this layer



- **For a batch of training samples…**
- **For another batch**
  - Do an independent random sampling.



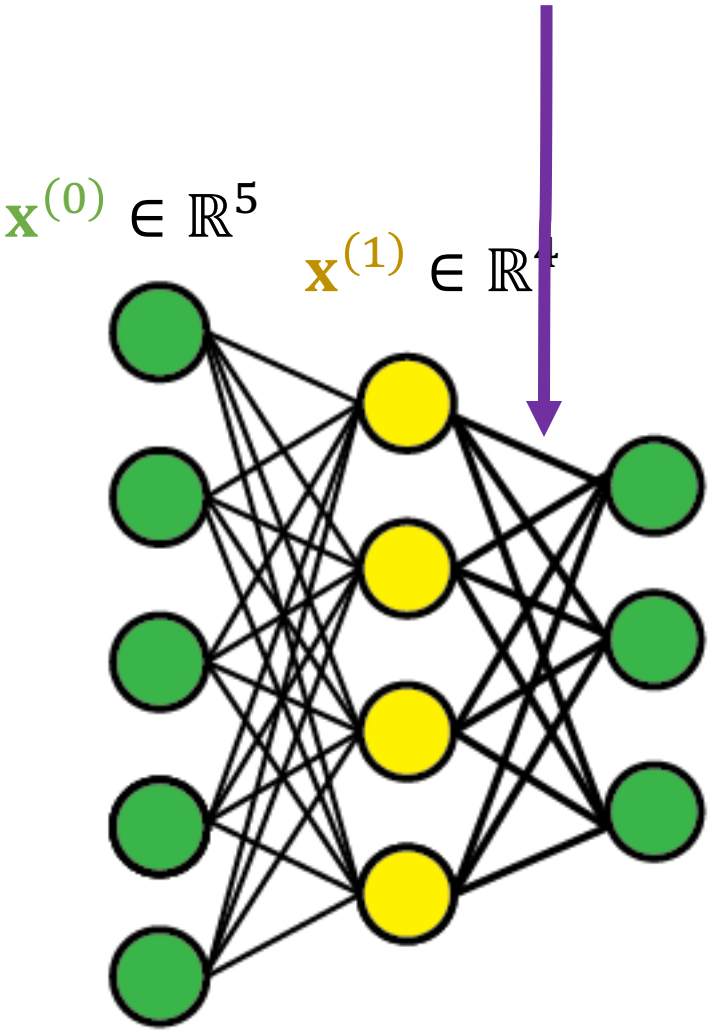
Dropout: Implementation



- **Input**: vector $\mathbf{x}^{(0)} \in \mathbb{R}^5$.
- $\mathbf{z}^{(1)} = \mathbf{W}^{(0)}\,\mathbf{x}^{(0)} \in \mathbb{R}^4$.
- $\mathbf{x}^{(1)} = \max\{\mathbf{0},\ \mathbf{z}^{(1)}\} \in \mathbb{R}^4$.
- $\mathbf{z}^{(2)} = \mathbf{W}^{(1)}\,\mathbf{x}^{(1)} \in \mathbb{R}^3$.
- **Output**: $\mathbf{z}^{(2)} \in \mathbb{R}^3$.

Dropout: Implementation

Regularize this layer



- **Input**: vector $\mathbf{x}^{(0)} \in \mathbb{R}^5$.
- $\mathbf{z}^{(1)} = \mathbf{W}^{(0)} \mathbf{x}^{(0)} \in \mathbb{R}^4$.
- $\mathbf{x}^{(1)} = \max\{\mathbf{0}, \mathbf{z}^{(1)}\} \in \mathbb{R}^4$.
- Add a dropout layer
- $\mathbf{z}^{(2)} = \mathbf{W}^{(1)} \tilde{\mathbf{x}}^{(1)} \in \mathbb{R}^3$.
- **Output**: $\mathbf{z}^{(2)} \in \mathbb{R}^3$.

- $\mathbf{m} \in \mathbb{R}^4$ is a random vector. (Each entry is 0 or 2, w.p. 50%.)
- Apply $\mathbf{m}$ to $\mathbf{x}^{(1)}$:

$$\tilde{\mathbf{x}}^{(1)} = \mathbf{m} \circ \mathbf{x}^{(1)}.$$

" $\circ$ " is entrywise multiplication.

Keras's Dropout Layer

- Dropout only before the 1st dense layer to regularize the 1st dense layer.
- Because the 1st dense layer has too many trainable parameters.

```
model = models.Sequential()
model.add(layers.Conv2D(32, (3, 3), activation='relu',
                        input_shape=(150, 150, 3)))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(128, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(128, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Flatten())
model.add(layers.Dropout(0.5))
model.add(layers.Dense(512, activation='relu'))
model.add(layers.Dense(1, activation='sigmoid'))

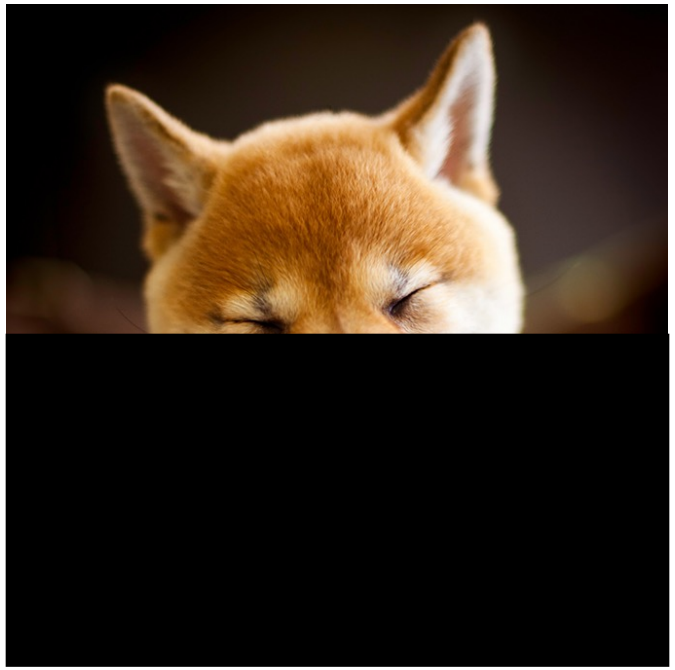
model.compile(loss='binary_crossentropy',
              optimizer=optimizers.RMSprop(lr=1e-4),
              metrics=['acc'])
```

Why Does Dropout Work?

- In training, dropout forces the network to make decision based on part of the features.







Why Does Dropout Work?

- In training, dropout forces the network to make decision based on part of the features.

- Dropout is a regularization [1].
  - Alleviate overfitting.
  - Like the L1 and L2 norm regularizations.
  - But dropout is empirically better.

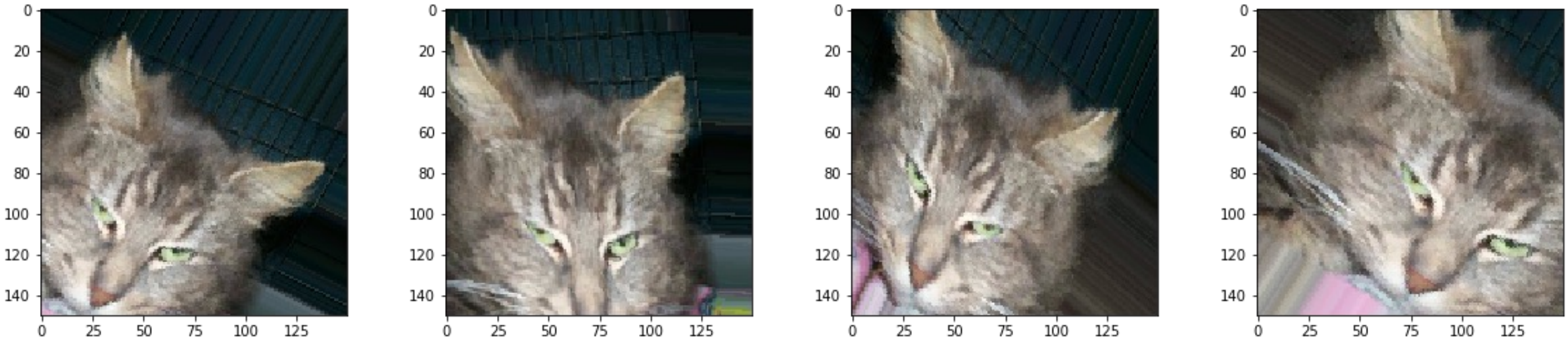
Reference:

1. Wager, Wang, & Liang. Dropout Training as Adaptive Regularization. In *NIPS*, 2013.

Trick 2: Data Augmentation

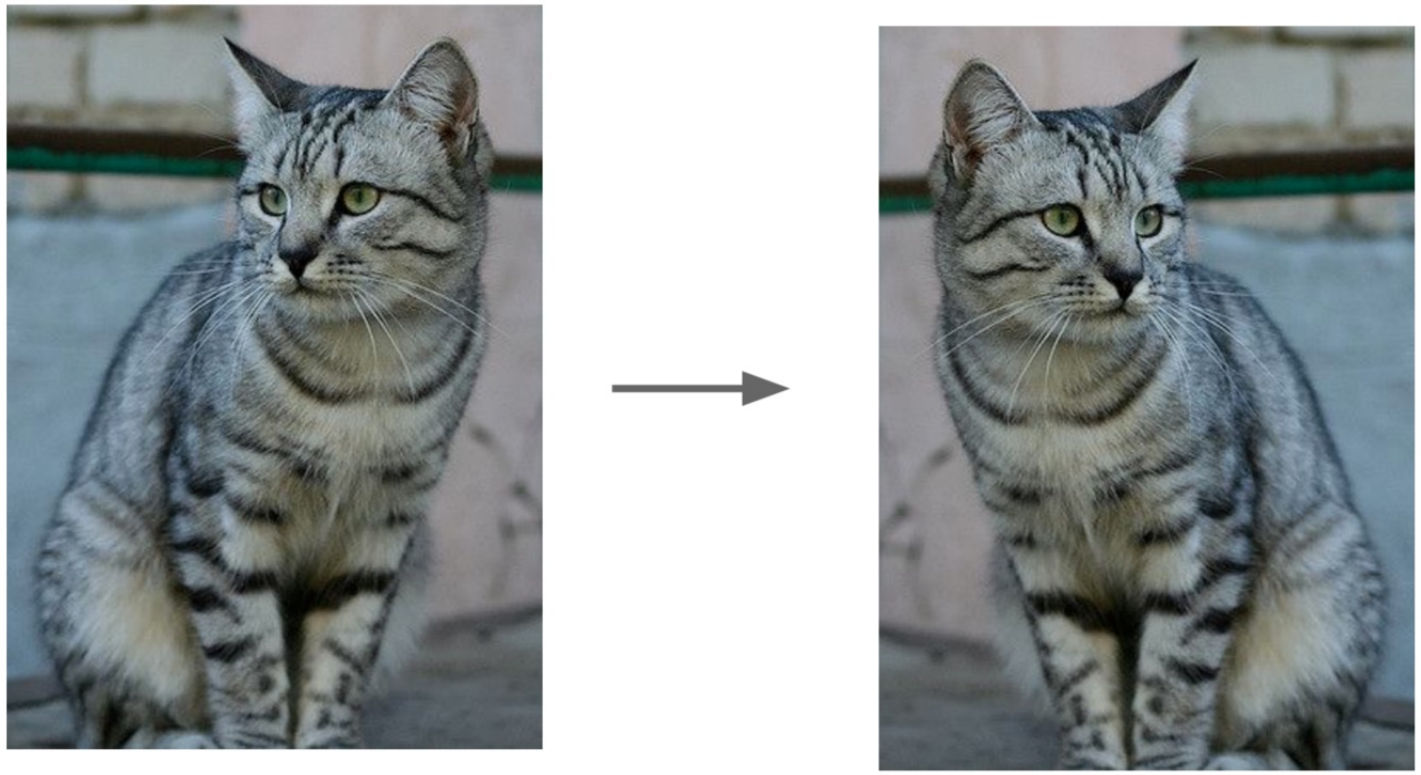
Data Augmentation

- Data augmentation: generating more training samples from existing training data.
- E.g., flip, rotation, crop, shift, add random noise.



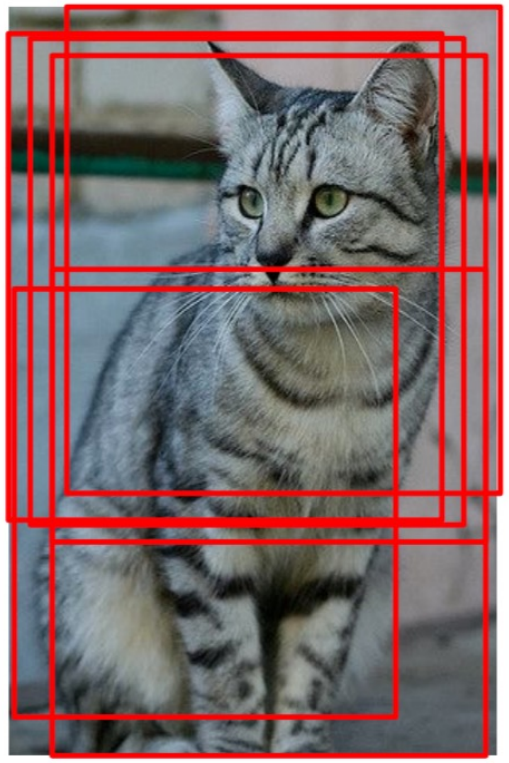
Data Augmentation: Examples

Horizontal Flips



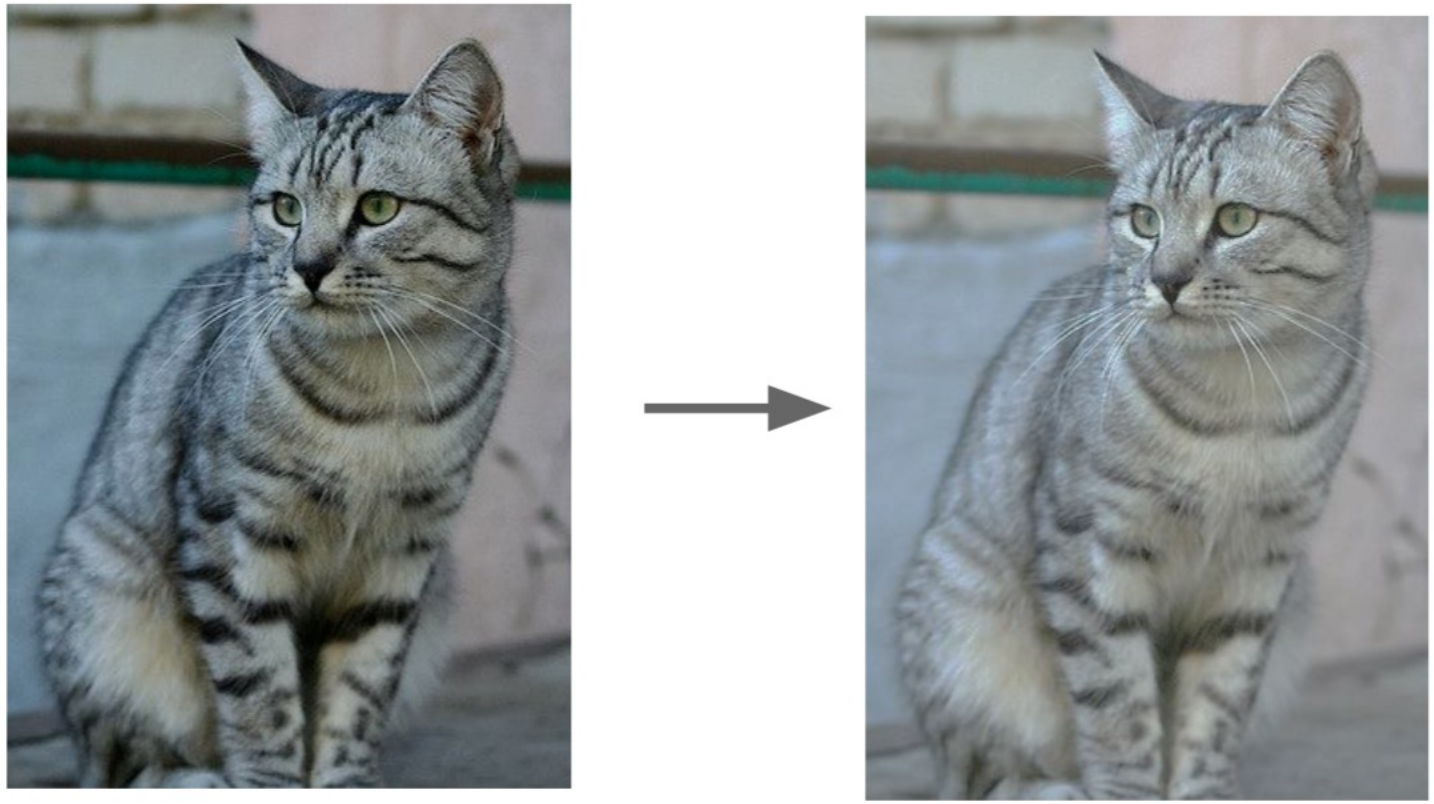
Data Augmentation: Examples

Random crops and scales



Data Augmentation: Examples

Color Jitter (randomize contrast and brightness)



Take-Home Message

- To train CNN for images, always use data augmentation.
  - It gives you more data for free!
  - Use data augmentation for only training samples.
  - Don’t use it for validation and test samples!!!

Take-Home Message

- To train CNN for images, always use data augmentation.
  - It gives you more data for free!
  - Use data augmentation for only training samples.
  - Don’t use it for validation and test samples!!!

- If a layer has too many parameters, place a dropout layer before it.
  - Dropout randomly masks the inputs.
  - Dropout is regularization.
  - Regularization alleviates overfitting.
  - The training becomes a little slower.

Trick 3: Pretrain

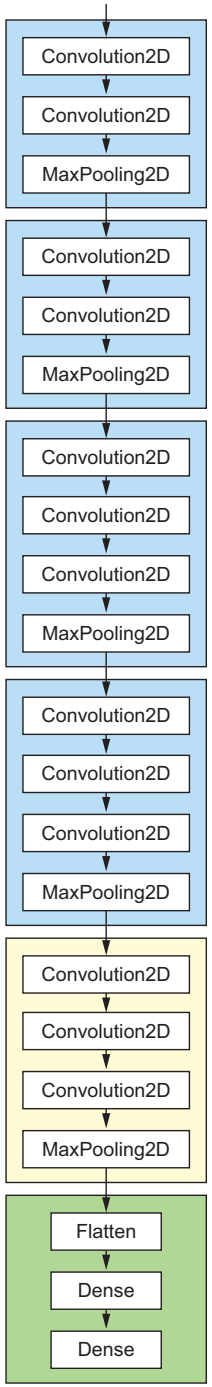
Train a Deep Neural Network?

```
model = models.Sequential()
model.add(layers.Conv2D(32, (3, 3), activation='relu',
                        input_shape=(150, 150, 3)))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(128, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(128, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Flatten())
model.add(layers.Dropout(0.5))
model.add(layers.Dense(512, activation='relu'))
model.add(layers.Dense(1, activation='sigmoid'))

model.compile(loss='binary_crossentropy',
              optimizer=optimizers.RMSprop(lr=1e-4),
              metrics=['acc'])
```

- We have trained a neural net with 4 Conv Layers and 2 FC Layers.
- Relatively shallow.

Train a Deep Neural Network?



the VGG16 network

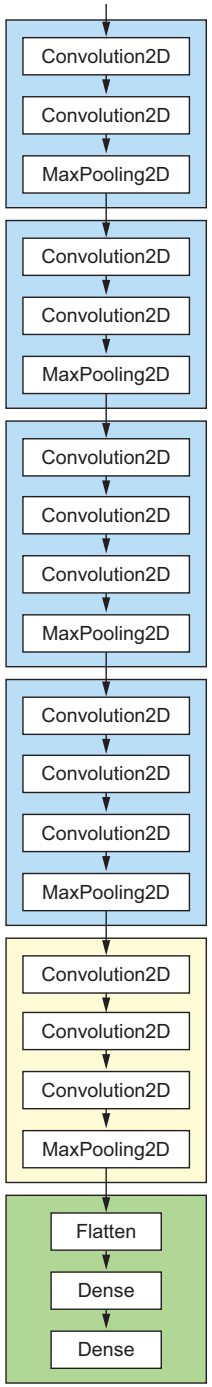
Can we train a deep neural network?

- It is hard!
  - The number of parameters is huge.
  - The deep network has a huge capacity.
  - We have merely $2K$ training samples.
- Naively training a deep network will results in overfitting, surely.

Solution: pretrain

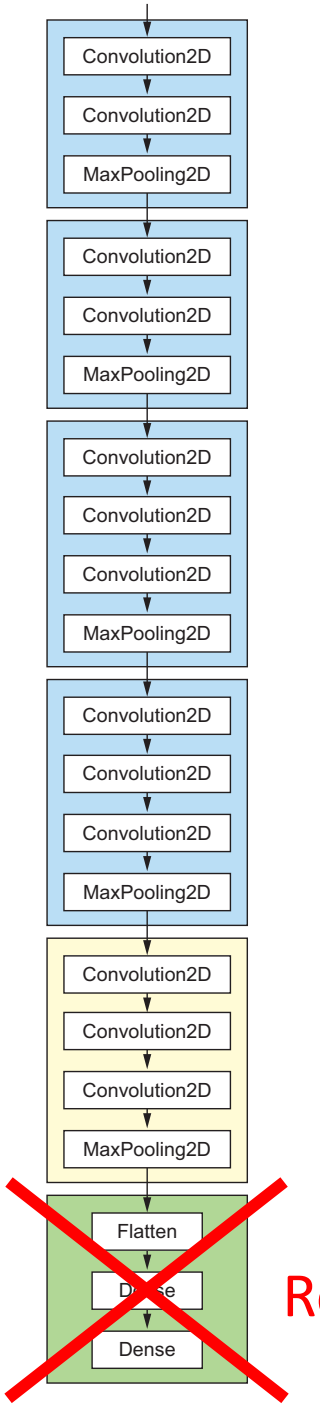
Pretrain

1. Pre-train a deep net on large-scale dataset, e.g., ImageNet ($14M$ images with labels).



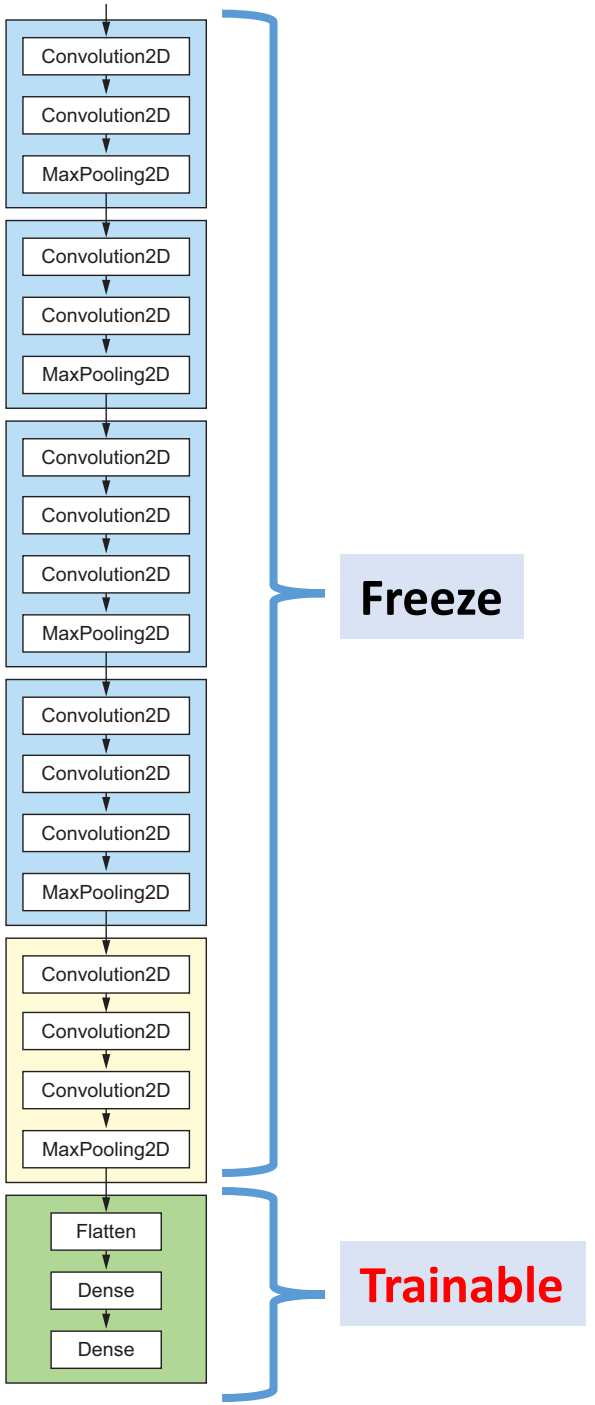
the VGG16 network

Pretrain



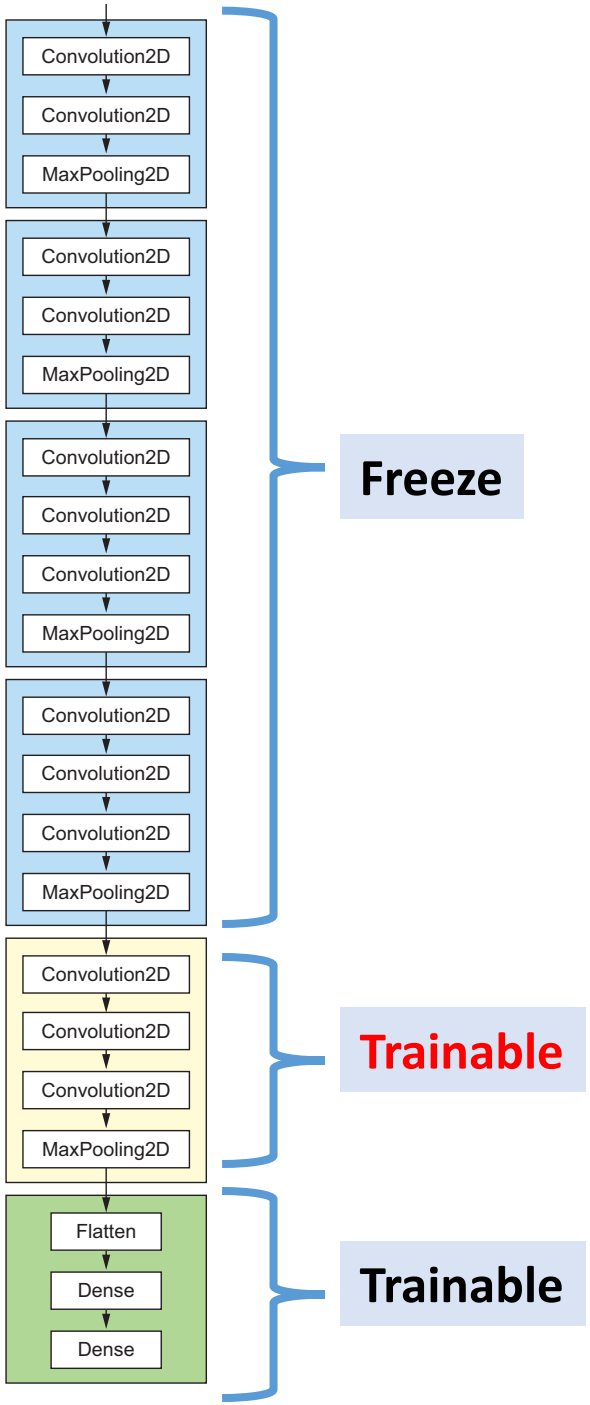
Remove the top layers

1. Pre-train a deep net on large-scale dataset, e.g., ImageNet ($14M$ images with labels).
2. Remove the top layers. *Why?*
   - Different output shapes and activation functions.
   - New classifier specialized for the new task.



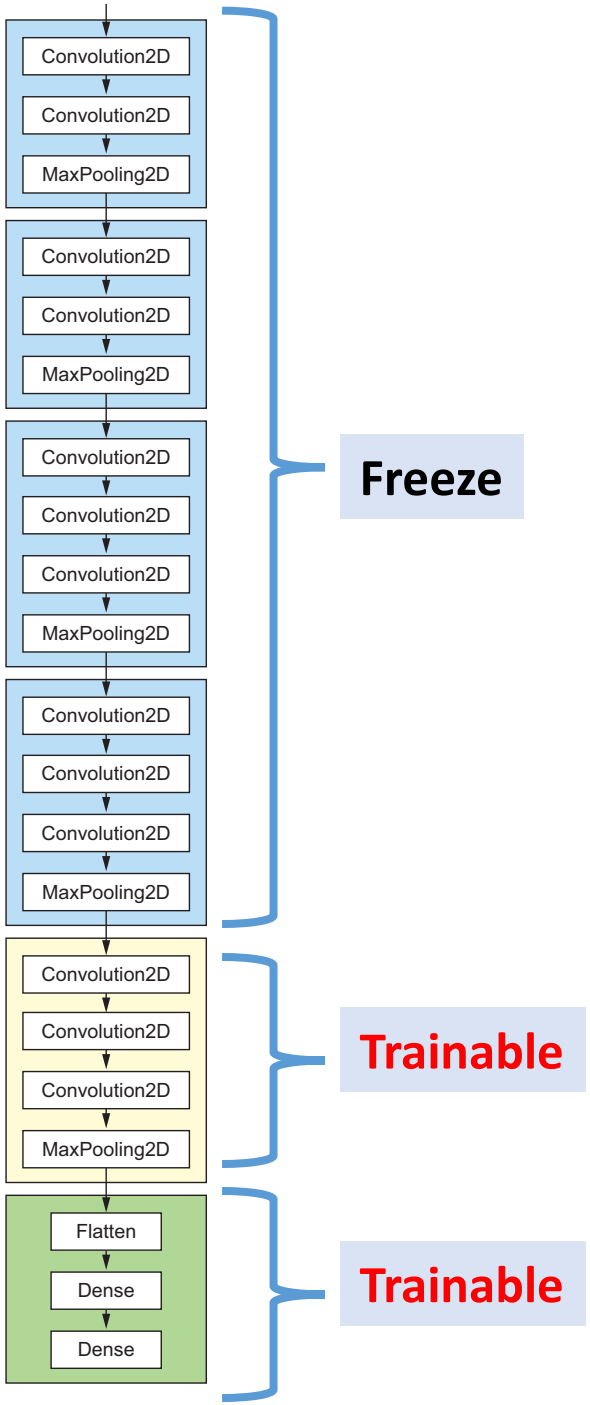
Pretrain

1. Pre-train a deep net on large-scale dataset, e.g., ImageNet ($14M$ images with labels).
2. Remove the top layers.
3. Build new top layers (randomly initialized).
4. Freeze the base layers; Train the top layers.



Pretrain

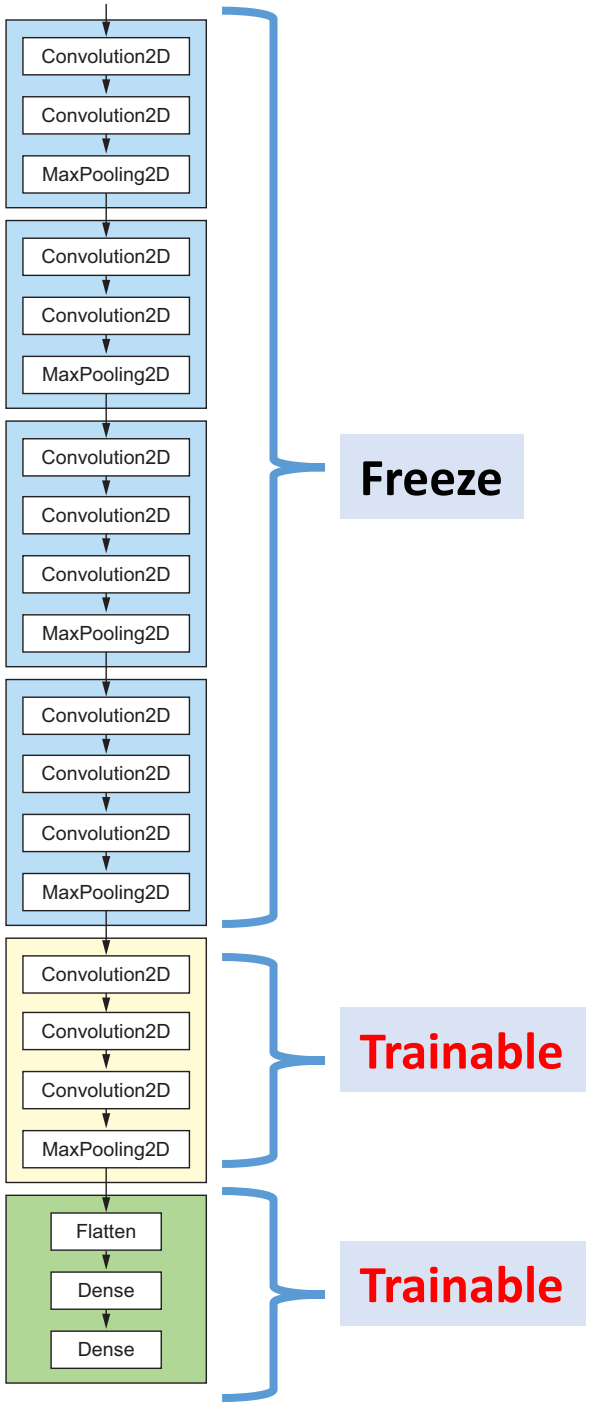
1. Pre-train a deep net on large-scale dataset, e.g., ImageNet ($14M$ images with labels).
2. Remove the top layers.
3. Build new top layers (randomly initialized).
4. Freeze the base layers; Train the top layers.
5. Optional: **Fine-tune** the top Conv Layers.



Pretrain

1. Pre-train a deep net on large-scale dataset, e.g., ImageNet ($14M$ images with labels).
2. Remove the top layers.
3. Build new top layers (randomly initialized).
4. Freeze the base layers; Train the top layers.
5. Optional: **Fine-tune** the top Conv Layers.

Question: Can Steps 4 & 5 be merged?



Pretrain

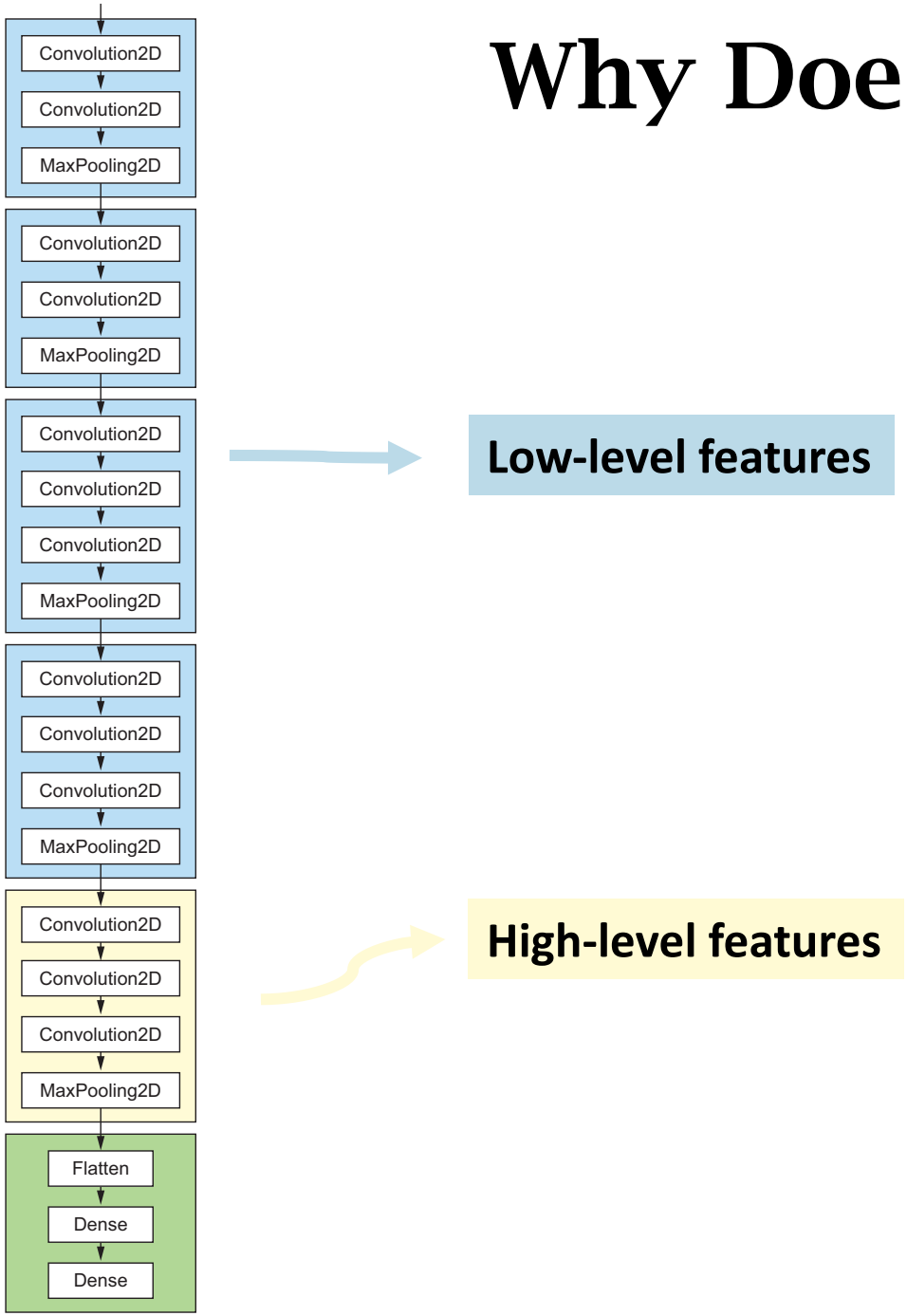
No. Merging Steps 4 & 5 is a bad idea.

- If the top layers are random, the initial gradients are large.
- The large gradient will destroy the Conv layers.

Thus, train the Conv layer after training the top layers.

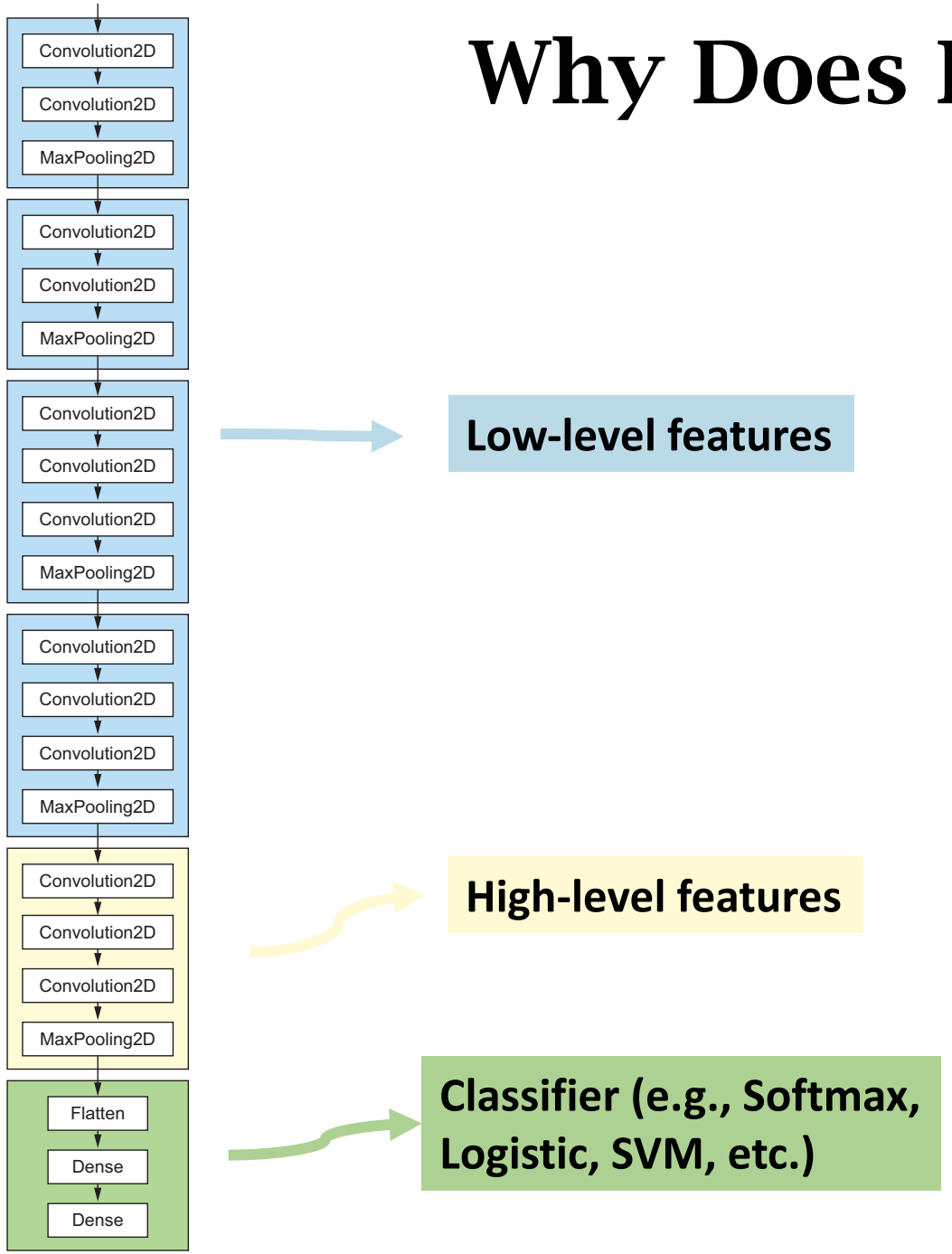
Question: Can Steps 4 & 5 be merged?

Why Does Pretraining Work?



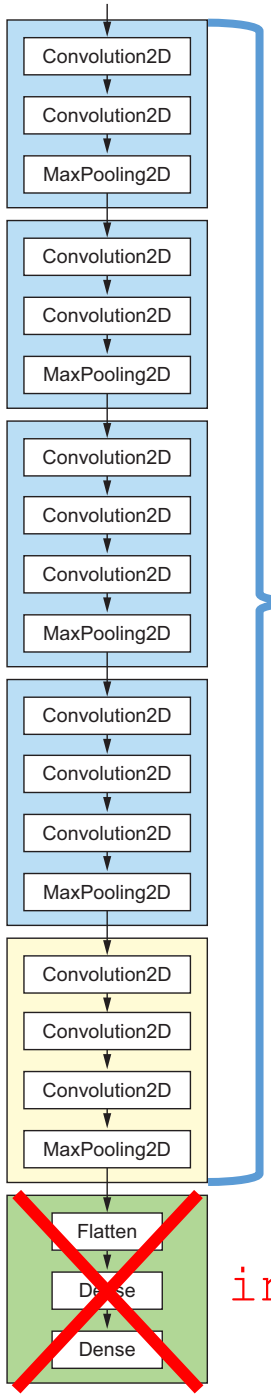
- The ***Conv Layers*** are for *feature extraction*.
  - The **low-level feature** (edges, shapes, patterns, etc.) learned from ImageNet are effective to other image problems.
  - The **high-level features** learned from ImageNet are useful, but less effective.

Why Does Pretraining Work?



- The ***Conv Layers*** are for *feature extraction*.
  - The **low-level feature** (edges, shapes, patterns, etc.) learned from ImageNet are effective to other image problems.
  - The **high-level features** learned from ImageNet are useful, but less effective.
- Think of the ***FC layers*** as a classifier which takes the extracted features as input.
- *Less* trainable parameters, *less* prone to overfitting.

Use a VGG16 Net Pretrained on ImageNet

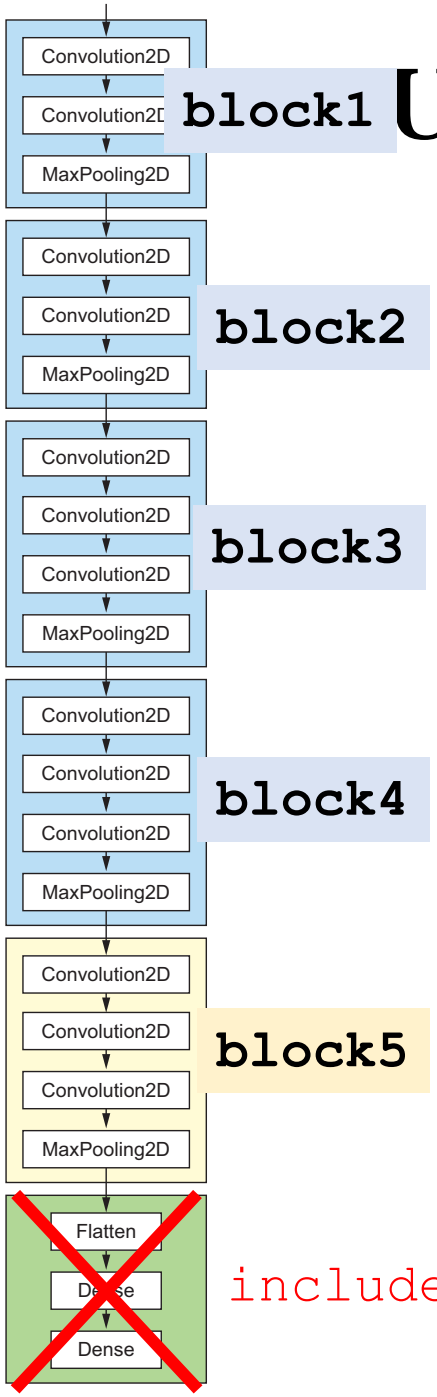



```
from keras.applications import VGG16

conv_base = VGG16(weights='imagenet',
                  include_top=False,
                  input_shape=(150, 150, 3))

conv_base.summary()
```

Use a VGG16 Net Pretrained on ImageNet

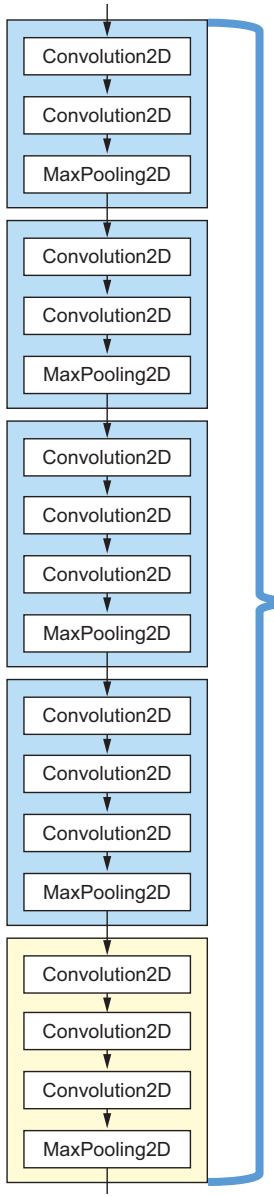


include_top=False

Layer (type)	Output Shape	Param #
input_2 (InputLayer)	(None, 150, 150, 3)	0
block1_conv1 (Conv2D)	(None, 150, 150, 64)	1792
block1_conv2 (Conv2D)	(None, 150, 150, 64)	36928
block1_pool (MaxPooling2D)	(None, 75, 75, 64)	0
block2_conv1 (Conv2D)	(None, 75, 75, 128)	73856
block2_conv2 (Conv2D)	(None, 75, 75, 128)	147584
block2_pool (MaxPooling2D)	(None, 37, 37, 128)	0
block3_conv1 (Conv2D)	(None, 37, 37, 256)	295168
block3_conv2 (Conv2D)	(None, 37, 37, 256)	590080
block3_conv3 (Conv2D)	(None, 37, 37, 256)	590080
block3_pool (MaxPooling2D)	(None, 18, 18, 256)	0
block4_conv1 (Conv2D)	(None, 18, 18, 512)	1180160
block4_conv2 (Conv2D)	(None, 18, 18, 512)	2359808
block4_conv3 (Conv2D)	(None, 18, 18, 512)	2359808
block4_pool (MaxPooling2D)	(None, 9, 9, 512)	0
block5_conv1 (Conv2D)	(None, 9, 9, 512)	2359808
block5_conv2 (Conv2D)	(None, 9, 9, 512)	2359808
block5_conv3 (Conv2D)	(None, 9, 9, 512)	2359808
block5_pool (MaxPooling2D)	(None, 4, 4, 512)	0

Total params: 14,714,688
Trainable params: 14,714,688
Non-trainable params: 0

Use a VGG16 Net Pretrained on ImageNet

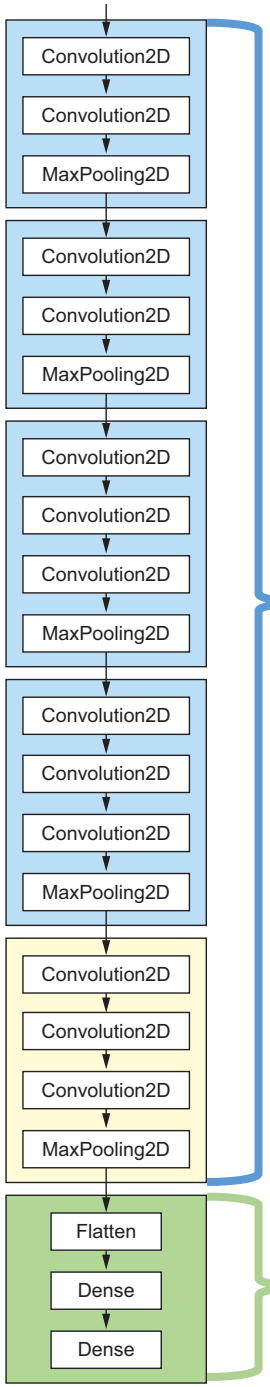


Base
(trainable)

```
from keras import models
from keras import layers

model = models.Sequential()
model.add(conv_base)
```

Use a VGG16 Net Pretrained on ImageNet



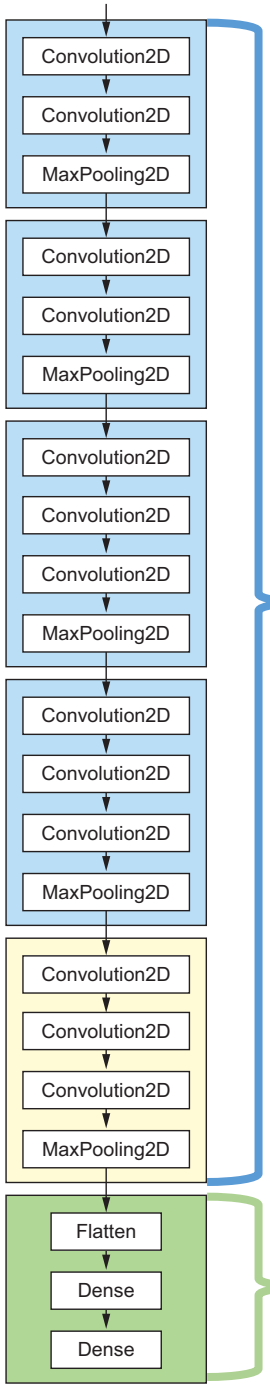
Base (trainable)

New Top (trainable)

```
from keras import models
from keras import layers

model = models.Sequential()
model.add(conv_base)
model.add(layers.Flatten())
model.add(layers.Dense(256, activation='relu'))
model.add(layers.Dense(1, activation='sigmoid'))
```

Use a VGG16 Net Pretrained on ImageNet



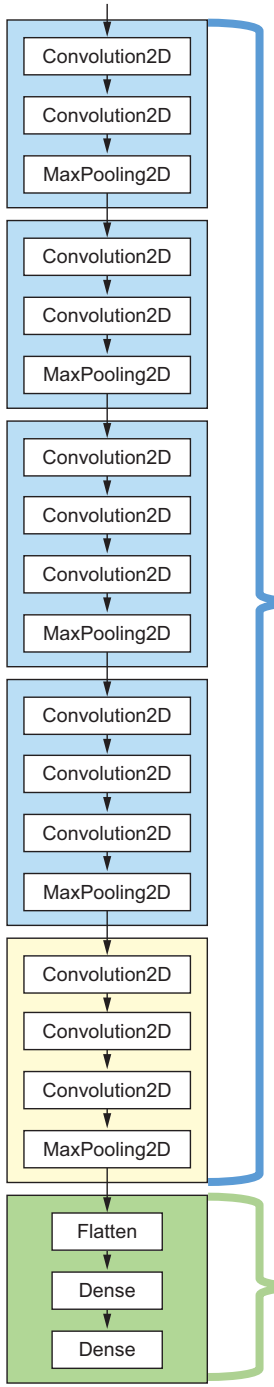
Base (trainable)

New Top (trainable)

Layer (type)	Output Shape	Param #
vgg16 (Model)	(None, 4, 4, 512)	14714688
flatten_1 (Flatten)	(None, 8192)	0
dense_1 (Dense)	(None, 256)	2097408
dense_2 (Dense)	(None, 1)	257

Total params: 16,812,353
Trainable params: 16,812,353
Non-trainable params: 0

Use a VGG16 Net Pretrained on ImageNet



Base (freeze)

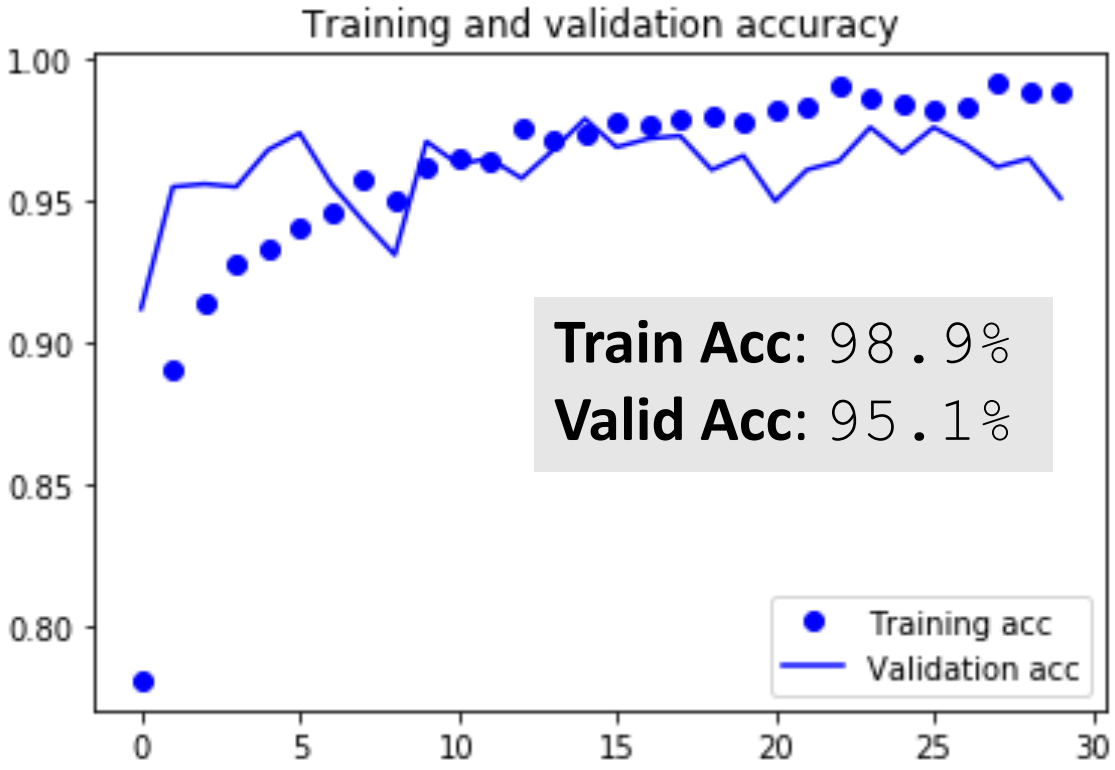
New Top (trainable)

```
conv_base.trainable = False
model.summary()
```

```
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
vgg16 (Model)                (None, 4, 4, 512)         14714688
_________________________________________________________________
flatten_1 (Flatten)          (None, 8192)              0
_________________________________________________________________
dense_1 (Dense)              (None, 256)               2097408
_________________________________________________________________
dense_2 (Dense)              (None, 1)                 257
=================================================================
Total params: 16,812,353
Trainable params: 2,097,665
Non-trainable params: 14,714,688
```

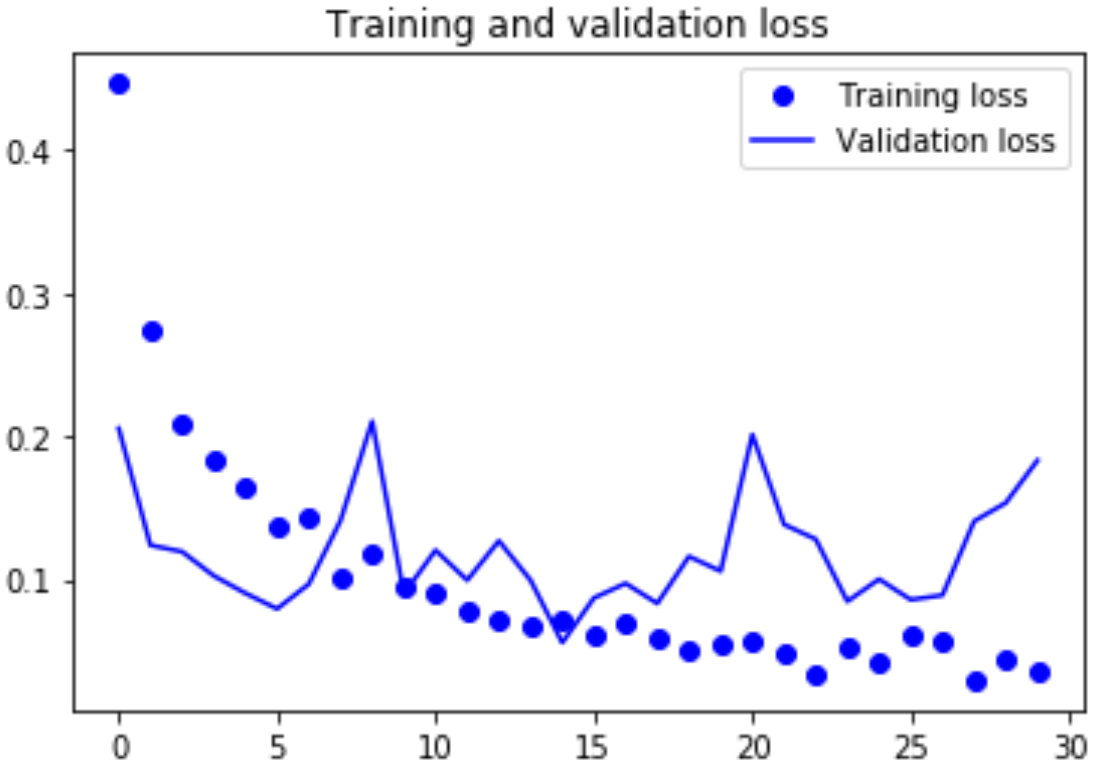
After Training the New Top

accuracy against ***epochs***

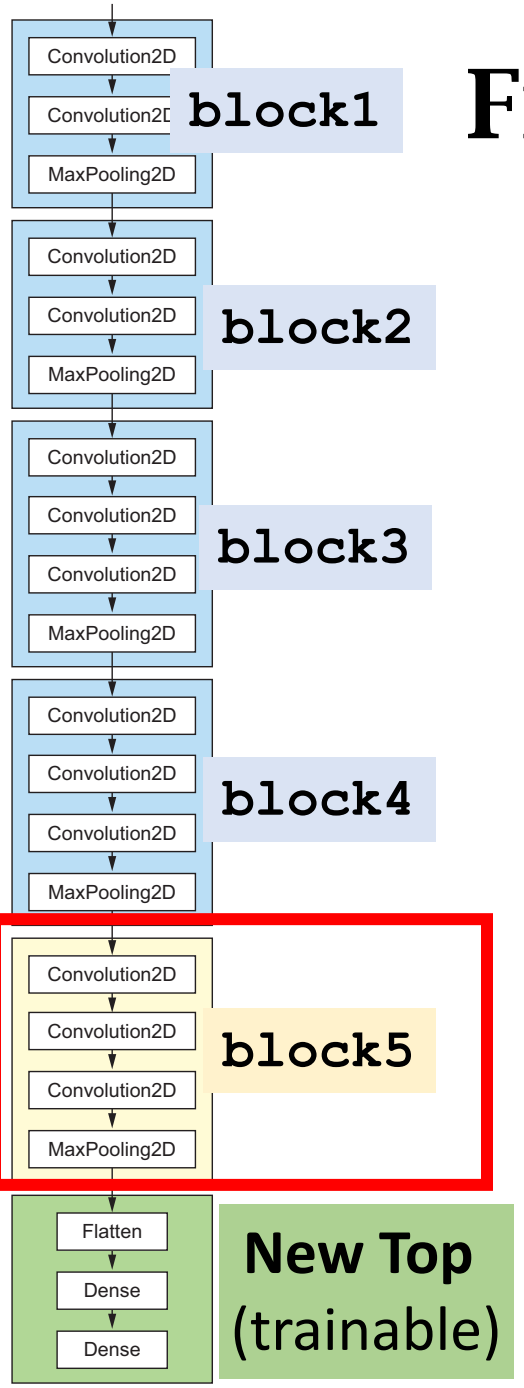


Train Acc: 98.9%
Valid Acc: 95.1%

loss against ***epochs***



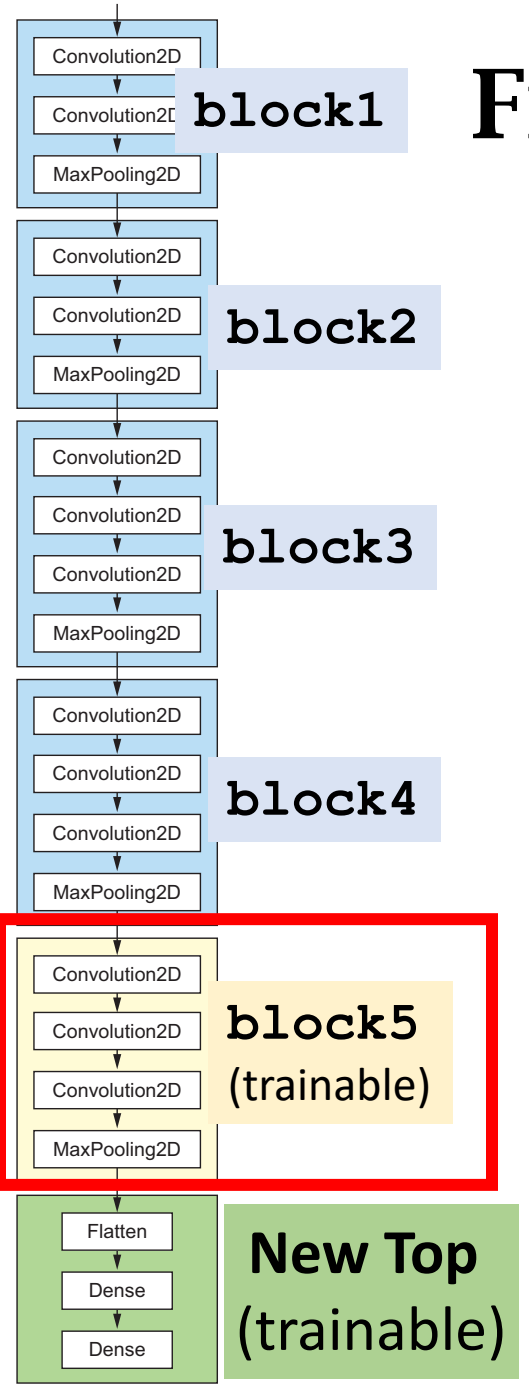
Fine Tuning the Top Conv Layers



Layer (type)	Output Shape	Param #
input_2 (InputLayer)	(None, 150, 150, 3)	0
block1_conv1 (Conv2D)	(None, 150, 150, 64)	1792
block1_conv2 (Conv2D)	(None, 150, 150, 64)	36928
block1_pool (MaxPooling2D)	(None, 75, 75, 64)	0
block2_conv1 (Conv2D)	(None, 75, 75, 128)	73856
block2_conv2 (Conv2D)	(None, 75, 75, 128)	147584
block2_pool (MaxPooling2D)	(None, 37, 37, 128)	0
block3_conv1 (Conv2D)	(None, 37, 37, 256)	295168
block3_conv2 (Conv2D)	(None, 37, 37, 256)	590080
block3_conv3 (Conv2D)	(None, 37, 37, 256)	590080
block3_pool (MaxPooling2D)	(None, 18, 18, 256)	0
block4_conv1 (Conv2D)	(None, 18, 18, 512)	1180160
block4_conv2 (Conv2D)	(None, 18, 18, 512)	2359808
block4_conv3 (Conv2D)	(None, 18, 18, 512)	2359808
block4_pool (MaxPooling2D)	(None, 9, 9, 512)	0
block5_conv1 (Conv2D)	(None, 9, 9, 512)	2359808
block5_conv2 (Conv2D)	(None, 9, 9, 512)	2359808
block5_conv3 (Conv2D)	(None, 9, 9, 512)	2359808
block5_pool (MaxPooling2D)	(None, 4, 4, 512)	0

Total params: 14,714,688
Trainable params: 14,714,688
Non-trainable params: 0

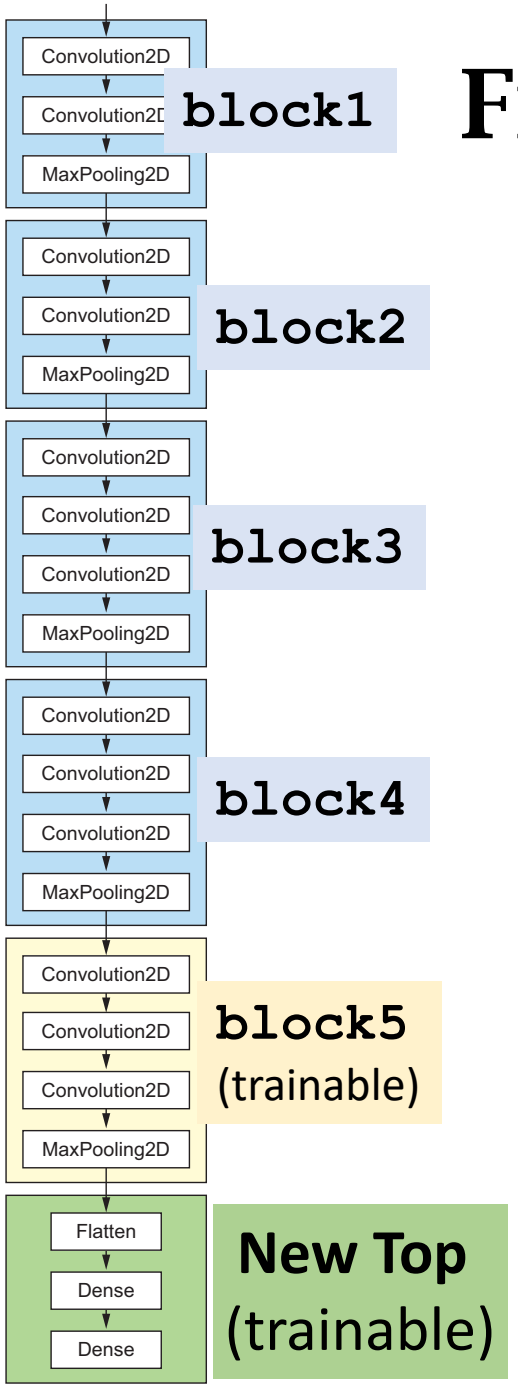
Fine Tuning the Top Conv Layers




```
trainable_layer_names = ['block5_conv1', 'block5_conv2',
                         'block5_conv3', 'block5_pool']
conv_base.trainable = True

for layer in conv_base.layers:
    if layer.name in trainable_layer_names:
        layer.trainable = True
    else:
        layer.trainable = False
```

Fine Tuning the Top Conv Layers




```
model.summary()
```

```
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
vgg16 (Model)                (None, 4, 4, 512)         14714688
_________________________________________________________________
flatten_1 (Flatten)          (None, 8192)              0
_________________________________________________________________
dense_1 (Dense)              (None, 256)               2097408
_________________________________________________________________
dense_2 (Dense)              (None, 1)                 257
=================================================================
Total params: 16,812,353
Trainable params: 9,177,089
Non-trainable params: 7,635,264
_________________________________________________________________
```

Fine Tuning the Top Conv Layers

Re-compile before training

```
model.compile(loss='binary_crossentropy',
              optimizer=optimizers.RMSprop(lr=1e-5),
              metrics=['acc'])
```